Referência da API · Bibliotela

Todos os endpoints do servidor mock da Bibliotela (`server/`, versão 0.1.0), com verbo, formato da requisição e resposta esperada. Os exemplos foram **capturados do servidor rodando** (`dart run tool/capture.dart`), e não escritos à mão. O servidor segue a estrutura da API da Carteira (mesmas rotas de login, perfil e erro, mesmos cabeçalhos de simulação) e troca os lançamentos por itens da biblioteca.

Visão geral

Verbo	Rota	Autenticação	Sucesso
`GET`	`/health`	não	200
`POST`	`/auth/login`	não	200
`POST`	`/auth/logout`	sim	204
`GET`	`/me`	sim	200
`PUT`	`/me`	sim	200
`GET`	`/genres`	sim	200
`GET`	`/items`	sim	200
`GET`	`/items/:id`	sim	200
`POST`	`/items`	sim	201
`PUT`	`/items/:id`	sim	200
`DELETE`	`/items/:id`	sim	204
`GET`	`/summary`	sim	200
`POST`	`/reset`	sim	204

Regras gerais

- **Base URL:** `http://localhost:8080`. No emulador Android, `http://10.0.2.2:8080`. No Codespaces, o endereço público da porta 8080.
- **Formato:** corpo e resposta em JSON, com `Content-Type: application/json; charset=utf-8`. Respostas 204 não têm corpo.
- **Autenticação:** as rotas protegidas exigem `Authorization: Bearer <token>`, com o token devolvido pelo login. Uma rota que não existe também exige token (sem ele, 401; com ele, 404).
- **Dados por sessão:** cada login recebe uma cópia própria do seed. O que uma sessão altera, outra não vê, e `POST /reset` devolve a sessão ao seed.
- **Datas de leitura:** `startedAt` e `finishedAt` são só a data, `YYYY-MM-DD`, sem hora nem fuso. Elas dizem o dia em que a pessoa começou ou terminou, e não um instante: com hora em UTC, terminar um livro às 23h viraria o dia seguinte.
- **Data de criação:** `addedAt` é ISO 8601 em UTC (`2026-07-30T18:20:00Z`) e é gerada pelo servidor.
- **Nota:** inteiro de 1 a 10. O app mostra em estrelas dividindo por 2, com meia estrela (8 = 4 estrelas, 9 = 4,5). O servidor guarda só o inteiro.
- **Campos opcionais** vêm sempre, com `null` quando vazios. Campos que não pertencem ao tipo do item também vêm `null`.
- **Textos:** texto vazio (ou só espaços) vira `null` em `genre`, `note`, `author` e `avatarUrl`; `title` perde os espaços das pontas.

- **Ids:** `it_0001`, `it_0002`... gerados em sequência por sessão. Um id apagado não é reaproveitado.
- **Parâmetros de consulta vazios** (`?genre=`) são tratados como ausentes.
- **Latência:** de 300 a 800 ms por padrão (`LATENCY_MIN_MS` e `LATENCY_MAX_MS`). O cabeçalho `X-Delay-Ms` fixa o valor de uma requisição.
- **CORS:** qualquer origem é aceita, e `OPTIONS` responde 204.
- **Credenciais de teste:** `ana@bibliotela.dev` / `123456` e `bruno@bibliotela.dev` / `123456`.

Modelos

user

```
{
  "id": "usr_01",
  "name": "Ana Ribeiro",
  "email": "ana@bibliotela.dev",
  "avatarUrl": null,
  "createdAt": "2024-03-11T09:00:00Z"
}
```

item

```
{
  "id": "it_0001",
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null,
  "addedAt": "2026-07-30T18:20:00Z"
}
```

Campo	Tipo	Regra
`type`	`string`	`book`, `series` ou `movie`
`title`	`string`	não vazio; espaços nas pontas são removidos
`genre`	`string?`	um dos valores de `GET /genres`; texto vazio vira `null`
`status`	`string`	`want`, `in_progress` ou `done` (ver tabela abaixo)
`startedAt`	`string?`	`YYYY-MM-DD`; regra por status, abaixo
`finishedAt`	`string?`	`YYYY-MM-DD`; regra por status, abaixo
`rating`	`integer?`	de 1 a 10; só vale com status `done`
`note`	`string?`	texto livre; texto vazio vira `null`
`pages`	`integer?`	só em `book`; maior que zero
`author`	`string?`	só em `book`; texto vazio vira `null`
`seasons`	`integer?`	só em `series`; maior que zero
`durationMinutes`	`integer?`	só em `movie`; maior que zero
`addedAt`	`string`	gerado pelo servidor; ignorado se vier no corpo

Regras por status:

status	startedAt	finishedAt	rating
`want`	null	null	null
`in_progress`	obrigatório	null	null
`done`	opcional	obrigatório	opcional

- `startedAt` não pode ser depois de `finishedAt`.
- **O que não vale é descartado:** valores que não valem para o status ou para o tipo (uma `rating` num item `want`, `pages` num filme) voltam `null` na resposta, sem erro. Faltar o que é obrigatório, ou mandar um valor inválido para um campo que vale, dá **422**.
- Os textos de status mudam por tipo só na tela: `want`, `in_progress` e `done` são “Quero ler / Lendo / Lido” para livro e “Quero assistir / Assistindo / Assistido” para filme e série.

Erro

Toda falha tem o mesmo formato. Em 422, o objeto ganha `fields`, com uma mensagem por campo inválido:

```
{
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": {
      "title": "must not be empty",
      "genre": "must be one of the values from /genres",
      "startedAt": "must not be after finishedAt",
      "rating": "must be an integer from 1 to 10",
      "pages": "must be greater than zero"
    }
  }
}
```

Status	code	Quando
400	`bad_request`	corpo não é JSON válido, ou não é um objeto JSON
401	`invalid_credentials`	e-mail ou senha errados no login
401	`unauthorized`	token ausente, desconhecido ou já encerrado
401	`token_expired`	só com `X-Fail: expired`
404	`not_found`	item ou rota inexistente
422	`validation`	campos ou parâmetros inválidos
500	`internal`	erro inesperado ou `X-Fail: 500`

O `message` do 422 muda conforme a rota: `Invalid item` (itens), `Invalid login`, `Invalid profile` e `Invalid query` (parâmetros de consulta).

Saúde

`GET /health`

Verifica se o servidor está no ar. É pública.

Requisição:

```
GET /health
```

Resposta 200:

```
{
  "status": "ok",
  "version": "0.1.0"
}
```

Autenticação

`POST /auth/login`

Autentica o usuário e cria uma sessão com dados próprios. É pública.

Requisição:

```
POST /auth/login
Content-Type: application/json

{
  "email": "ana@bibliotela.dev",
  "password": "123456"
}
```

Resposta 200:

```
{
  "token": "tok_r37k8q6plx8j0wrfblz6aojz",
  "user": {
    "id": "usr_01",
    "name": "Ana Ribeiro",
    "email": "ana@bibliotela.dev",
    "avatarUrl": null,
    "createdAt": "2024-03-11T09:00:00Z"
  }
}
```

Erros:

Status	Exemplo	Resposta
400	JSON malformado	`{"error":{"code":"bad_request","message":"Malformed JSON body"}}`
401	senha errada	`{"error":{"code":"invalid_credentials","message":"Invalid email or password"}}`
422	campos vazios	`{"error":{"code":"validation","message":"Invalid login","fields":{"email":"must not be empty","password":"must not be empty"}}}`

`POST /auth/logout`

Encerra a sessão. Depois disso, o token passa a receber 401 `unauthorized`.

Requisição:

```
POST /auth/logout
Authorization: Bearer <token>
```

Resposta 204:

sem corpo.

Depois, com o mesmo token:

```
GET /me
Authorization: Bearer <token>
```

Resposta 401:

```
{
  "error": {
    "code": "unauthorized",
    "message": "Missing, unknown or ended token"
  }
}
```

Perfil

`GET /me`

Devolve o usuário da sessão. Erros: 401.

Requisição:

```
GET /me
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "id": "usr_01",
  "name": "Ana Ribeiro",
  "email": "ana@bibliotela.dev",
  "avatarUrl": null,
  "createdAt": "2024-03-11T09:00:00Z"
}
```

`PUT /me`

Atualiza nome e avatar. Os outros campos do usuário não mudam.

Campo	Regra
`name`	obrigatório, não vazio; espaços nas pontas são removidos
`avatarUrl`	string ou `null`; texto vazio vira `null`

Requisição:

```
PUT /me
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "Ana R.",
  "avatarUrl": null
}
```

Resposta 200:

```
{
  "id": "usr_01",
  "name": "Ana R.",
  "email": "ana@bibliotela.dev",
  "avatarUrl": null,
  "createdAt": "2024-03-11T09:00:00Z"
}
```

Erro 422:

```
PUT /me
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "   "
}

→ {
  "error": {
    "code": "validation",
    "message": "Invalid profile",
    "fields": {
      "name": "must not be empty"
    }
  }
}
```

Gêneros

`GET /genres`

Lista os gêneros aceitos em `genre`, em ordem alfabética, com “Outros” no fim. O app usa esta lista para montar o formulário e as barras do resumo, sem deixar os nomes escritos no código.

Requisição:

```
GET /genres
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "items": [
    "Ação",
    "Aventura",
    "Biografia",
    "Comédia",
    "Documentário",
    "Drama",
    "Fantasia",
    "Ficção científica",
    "Romance",
    "Suspense",
    "Terror",
    "Outros"
  ]
}
```

Itens

`GET /items`

Lista os itens da sessão, com filtros e paginação.

Parâmetro	Padrão	Regra
`year`	todos	`YYYY`; filtra pela data de leitura (ver abaixo)
`status`	todos	`want`, `in_progress` ou `done`
`type`	todos	`book`, `series` ou `movie`
`genre`	todos	um valor de `GET /genres`
`q`	—	busca em `title` e `author`, sem diferenciar maiúsculas
`page`	1	inteiro maior ou igual a 1
`pageSize`	20	inteiro de 1 a 100

- **Data de leitura:** é `finishedAt` nos itens `done` e `startedAt` nos `in_progress`. É ela que define o ano do filtro `year`, a ordenação e o mês em que o app agrupa a lista. Itens `want` não têm data de leitura: **aparecem em qualquer ano** (o filtro `year` os mantém), e o app os mostra num grupo “Na fila”.
- **Ordenação:** primeiro os itens com data de leitura, do mais recente para o mais antigo (empate: o mais novo em `addedAt`). Depois os `want`, do adicionado mais recentemente para o mais antigo. Assim o agrupamento por mês fica contínuo, e os da fila ficam sempre no fim.
- `total` conta os itens depois dos filtros. Uma página além do fim devolve `items: []`, não um erro.

Requisição (duas por página):

```
GET /items?pageSize=2
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "items": [
    {
      "id": "it_0039",
      "type": "movie",
      "title": "Titanic",
      "genre": "Romance",
      "status": "in_progress",
      "startedAt": "2026-10-30",
      "finishedAt": null,
      "rating": null,
      "note": null,
      "pages": null,
      "author": null,
      "seasons": null,
      "durationMinutes": 195,
      "addedAt": "2026-10-24T12:30:00Z"
    },
    {
      "id": "it_0009",
      "type": "book",
      "title": "Drácula",
      "genre": "Terror",
      "status": "done",
      "startedAt": "2026-10-01",
      "finishedAt": "2026-10-14",
      "rating": 8,
      "note": null,
      "pages": 488,
      "author": "Bram Stoker",
```

```
      "seasons": null,
      "durationMinutes": null,
      "addedAt": "2026-09-25T12:30:00Z"
    }
  ],
  "page": 1,
  "pageSize": 2,
  "total": 45
}
```

Com filtros:

```
GET /items?year=2026&status=done&type=book&pageSize=2
Authorization: Bearer <token>
```

Resposta 200 (9 livros concluídos em 2026, mostrando 2):

```
{
  "items": [
    {
      "id": "it_0009",
      "type": "book",
      "title": "Drácula",
      "genre": "Terror",
      "status": "done",
      "startedAt": "2026-10-01",
      "finishedAt": "2026-10-14",
      "rating": 8,
      "note": null,
      "pages": 488,
      "author": "Bram Stoker",
      "seasons": null,
      "durationMinutes": null,
      "addedAt": "2026-09-25T12:30:00Z"
    },
    {
      "id": "it_0001",
      "type": "book",
      "title": "Duna",
      "genre": "Ficção científica",
      "status": "done",
      "startedAt": "2026-08-12",
      "finishedAt": "2026-09-06",
      "rating": 9,
      "note": "Final inesperado",
      "pages": 680,
      "author": "Frank Herbert",
      "seasons": null,
      "durationMinutes": null,
      "addedAt": "2026-07-30T18:20:00Z"
    }
  ],
  "page": 1,
  "pageSize": 2,
  "total": 9
}
```

Busca por autor:

```
GET /items?q=tolkien
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "items": [
    {
      "id": "it_0024",
      "type": "book",
      "title": "O Hobbit",
      "genre": "Fantasia",
      "status": "done",
      "startedAt": "2025-07-05",
      "finishedAt": "2025-07-29",
      "rating": 8,
      "note": null,
      "pages": 336,
      "author": "J. R. R. Tolkien",
      "seasons": null,
      "durationMinutes": null,
      "addedAt": "2025-06-29T12:30:00Z"
    }
  ],
  "page": 1,
  "pageSize": 20,
  "total": 1
}
```

Página além do fim:

```
GET /items?page=99
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "items": [],
  "page": 99,
  "pageSize": 20,
  "total": 45
}
```

Erro 422:

```
GET /items?year=26&status=lendo&page=0
Authorization: Bearer <token>

→ {
  "error": {
    "code": "validation",
    "message": "Invalid query",
    "fields": {
      "year": "must be in the format YYYY",
      "status": "must be one of: want, in_progress, done",
      "page": "must be an integer greater than or equal to 1"
    }
  }
}
```

`GET /items/:id`

Devolve um item. Erros: 401 e 404.

Requisição:

```
GET /items/it_0001
Authorization: Bearer <token>
```

Resposta 200:

```
{
  "id": "it_0001",
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null,
  "addedAt": "2026-07-30T18:20:00Z"
}
```

Item inexistente:

```
GET /items/it_9999
Authorization: Bearer <token>

→ {"error":{"code":"not_found","message":"Item it_9999 not found"}}
```

`POST /items`

Cria um item. O servidor gera o `id` e o `addedAt`; se vierem no corpo, são ignorados.

Requisição:

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null
}
```

Resposta 201: o item criado, já normalizado.

```
{
  "id": "it_0046",
  "type": "book",
  "title": "Duna",
  "genre": "Ficção científica",
  "status": "done",
  "startedAt": "2026-08-12",
  "finishedAt": "2026-09-06",
  "rating": 9,
  "note": "Final inesperado",
  "pages": 680,
  "author": "Frank Herbert",
  "seasons": null,
  "durationMinutes": null,
  "addedAt": "2026-11-06T23:02:33Z"
}
```

O que não vale é descartado e os textos são normalizados:

Requisição (filme “quero”, com nota, páginas e data que não valem):

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "movie",
  "title": "  Interestelar  ",
  "genre": "",
  "status": "want",
  "startedAt": "2026-10-01",
  "rating": 9,
  "pages": 300,
  "durationMinutes": 169
}
```

Resposta 201:

```
{
  "id": "it_0047",
  "type": "movie",
  "title": "Interestelar",
  "genre": null,
  "status": "want",
  "startedAt": null,
  "finishedAt": null,
  "rating": null,
  "note": null,
  "pages": null,
  "author": null,
  "seasons": null,
  "durationMinutes": 169,
  "addedAt": "2026-11-06T23:02:33Z"
}
```

Erros 422:

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "book",
  "title": "",
  "genre": "Nada",
  "status": "done",
  "startedAt": "2026-09-10",
  "finishedAt": "2026-09-01",
  "rating": 11,
  "pages": 0
}

→ {
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": {
      "title": "must not be empty",
      "genre": "must be one of the values from /genres",
      "startedAt": "must not be after finishedAt",
      "rating": "must be an integer from 1 to 10",
      "pages": "must be greater than zero"
    }
  }
}
```

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "livro",
  "status": "lendo"
}

→ {
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": {
      "type": "must be one of: book, series, movie",
      "title": "must not be empty",
      "status": "must be one of: want, in_progress, done"
    }
  }
}
```

Mensagens de `fields`, por campo:

Campo	Mensagem
`type`	`must be one of: book, series, movie`
`title`	`must not be empty` (ou `must be a string`)
`genre`	`must be one of the values from /genres`
`status`	`must be one of: want, in_progress, done`
`startedAt`	`is required when status is in_progress`; `must not be after finishedAt`
`finishedAt`	`is required when status is done`
`startedAt, finishedAt`	`must be a date in the format YYYY-MM-DD` (data inexistente, como 2026-02-30, também cai aqui)
`rating`	`must be an integer from 1 to 10`
`pages, seasons, durationMinutes`	`must be greater than zero` (ou `must be an integer`)
`note, author`	`must be a string`

Corpo malformado (400):

```
POST /items
Authorization: Bearer <token>
Content-Type: application/json

{oops

→ {"error":{"code":"bad_request","message":"Malformed JSON body"}}
```

`PUT /items/:id`

Substitui um item inteiro. O corpo tem o mesmo formato do POST, e o `id` do caminho sempre prevalece sobre o do corpo. O `addedAt` original é mantido.

Requisição: terminando uma série que estava em andamento.

```
PUT /items/it_0038
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "series",
  "title": "Severance",
  "genre": "Suspense",
  "status": "done",
  "startedAt": "2026-09-20",
  "finishedAt": "2026-09-28",
  "rating": 9,
  "note": null,
  "pages": null,
  "author": null,
  "seasons": 2,
  "durationMinutes": null
}
```

Resposta 200: o item atualizado (o `addedAt` continua o de antes).

```
{
  "id": "it_0038",
  "type": "series",
  "title": "Severance",
  "genre": "Suspense",
  "status": "done",
  "startedAt": "2026-09-20",
  "finishedAt": "2026-09-28",
  "rating": 9,
  "note": null,
  "pages": null,
  "author": null,
  "seasons": 2,
  "durationMinutes": null,
  "addedAt": "2026-09-14T21:40:00Z"
}
```

Erros: 400, 401, 422 (como no POST) e 404 quando o id não existe. **A validação vem antes:** um corpo inválido para um id inexistente recebe 422, e só um corpo válido recebe 404.

Corpo inválido, id inexistente:

```
PUT /items/it_9999
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": ""
}

→ {
  "error": {
    "code": "validation",
    "message": "Invalid item",
    "fields": {
      "type": "must be one of: book, series, movie",
      "title": "must not be empty",
      "status": "must be one of: want, in_progress, done"
    }
  }
}
```

Corpo válido, id inexistente:

```
PUT /items/it_9999
Authorization: Bearer <token>
Content-Type: application/json

{
  "type": "book",
  "title": "Qualquer",
  "status": "want"
}

→ {"error":{"code":"not_found","message":"Item it_9999 not found"}}
```

`DELETE /items/:id`

Remove um item. Resposta 204, sem corpo. Erros: 401 e 404 (inclusive ao apagar duas vezes o mesmo id).

Requisição:

```
DELETE /items/it_0046
Authorization: Bearer <token>
```

Resposta 204:

sem corpo.

Apagando o mesmo id de novo:

```
DELETE /items/it_0046
Authorization: Bearer <token>

→ {"error":{"code":"not_found","message":"Item it_0046 not found"}}
```

Resumo do ano

`GET /summary`

Soma o que foi concluído em um ano. Só entram itens `done` cujo `finishedAt` cai no ano pedido. O parâmetro `year` (`YYYY`) é opcional e, sem ele, vale o ano atual.

- `books.finished`: livros concluídos no ano. `books.pages`: soma de `pages` desses livros (livro sem `pages` conta 0).
- `movies.finished` e `movies.minutes`: filmes concluídos e soma de `durationMinutes`.
- `series.finished` e `series.seasons`: séries concluídas e soma de `seasons`.
- `averageRating`: média das notas dos itens concluídos no ano que têm nota, com uma casa decimal; `null` se nenhum tem.
- `byGenre`: itens concluídos no ano agrupados por gênero, do maior para o menor. `genre: null` é “Sem gênero”. **Em caso de empate**, o “sem gênero” vem primeiro e depois a ordem alfabética (no exemplo abaixo, `null` e Romance e Suspense empatam com 2).
- Um ano sem nada concluído devolve zeros, `averageRating: null` e `byGenre: []`.

Requisição:

```
GET /summary?year=2026
Authorization: Bearer <token>
```

Resposta 200 (Ana, 2026):


```
{
  "year": 2026,
  "books": {
    "finished": 9,
    "pages": 3024
  },
  "movies": {
    "finished": 8,
    "minutes": 1102
  },
  "series": {
    "finished": 4,
    "seasons": 11
  },
  "averageRating": 8.5,
  "byGenre": [
    {
      "genre": "Ficção científica",
      "count": 5
    },
    {
      "genre": "Drama",
      "count": 4
    },
    {
      "genre": "Comédia",
      "count": 3
    },
    {
      "genre": null,
      "count": 2
    },
    {
      "genre": "Romance",
      "count": 2
    },
    {
      "genre": "Suspense",
      "count": 2
    },
    {
      "genre": "Fantasia",
      "count": 1
    },
    {
      "genre": "Outros",
      "count": 1
    },
    {
      "genre": "Terror",
      "count": 1
    }
  ]
}
```


Ano sem nada concluído:

```
GET /summary?year=1999
Authorization: Bearer <token>

→ {
  "year": 1999,
  "books": {
    "finished": 0,
    "pages": 0
  },
  "movies": {
    "finished": 0,
```

```
    "minutes": 0
  },
  "series": {
    "finished": 0,
    "seasons": 0
  },
  "averageRating": null,
  "byGenre": []
}
```

Ano inválido (422):

```
GET /summary?year=26
Authorization: Bearer <token>

→ {
  "error": {
    "code": "validation",
    "message": "Invalid query",
    "fields": {
      "year": "must be in the format YYYY"
    }
  }
}
```

Utilitários

`POST /reset`

Devolve os dados da sessão (itens e perfil) ao estado do seed. O token continua válido. Resposta 204. Erros: 401.

Requisição:

```
POST /reset
Authorization: Bearer <token>
```

Resposta 204:

sem corpo.

Cabeçalhos de simulação

Valem em qualquer rota e são avaliados **antes** da autenticação.

Cabeçalho	Efeito
`X-Delay-Ms: 3000`	usa exatamente esta latência, em ms
`X-Fail: 500`	500 com `{"error":{"code":"internal","message":"Simulated failure"}}`
`X-Fail: expired`	401 com `{"error":{"code":"token_expired","message":"Token has expired"}}`
`X-Fail: timeout`	segura a requisição por 60 s e responde 504 em texto puro

X-Fail: 500

```
GET /health
X-Fail: 500
```

Resposta 500:

```
{
  "error": {
    "code": "internal",
    "message": "Simulated failure"
  }
}
```

X-Fail: expired (funciona até sem token):

```
GET /me
X-Fail: expired
```

Resposta 401:

```
{
  "error": {
    "code": "token_expired",
    "message": "Token has expired"
  }
}
```

X-Fail: timeout

```
GET /health
X-Fail: timeout
```

Resposta 504:

```
Content-Type: text/plain; charset=utf-8

Gateway Timeout
```

Na captura, o timeout segurou a requisição por 505 ms, porque o script usa um limite de 0,5 s só para não esperar. No servidor normal são 60 s.

X-Delay-Ms: 250

```
GET /health
X-Delay-Ms: 250
```

Resposta 200:

```
{
  "status": "ok",
  "version": "0.1.0"
}
```

Levou 252 ms, contra 1 a 3 ms das outras chamadas (a captura roda sem a latência padrão).

Rota inexistente

Com um token válido, uma rota que não existe responde 404. Sem token, 401.

Com token:

```
GET /nada
Authorization: Bearer <token>
```

Resposta 404:

```
{
  "error": {
    "code": "not_found",
    "message": "Route /nada not found"
  }
}
```

Sem token:

```
GET /nada
```

Resposta 401:

```
{
  "error": {
    "code": "unauthorized",
    "message": "Missing, unknown or ended token"
  }
}
```

CORS

`OPTIONS` responde 204 em qualquer rota, e todas as respostas trazem os cabeçalhos abaixo:

```
OPTIONS /items

HTTP 204
access-control-allow-origin: *
access-control-allow-methods: GET, POST, PUT, DELETE, OPTIONS
access-control-allow-headers: Authorization, Content-Type, X-Delay-Ms, X-Fail
access-control-max-age: 86400
```

Seed

- **Usuários** (senha `123456` para todos): `usr_01` Ana Ribeiro (`ana@bibliotela.dev`) e `usr_02` Bruno Costa (`bruno@bibliotela.dev`).
- **Ana:** 45 itens. 14 concluídos em 2025, 21 concluídos em 2026, 4 em andamento e 6 na fila. **Bruno:** 15 itens. 4 concluídos em 2025, 6 em 2026, 2 em andamento e 3 na fila.
- **Variedade de propósito:** os três tipos, os três status, itens com e sem `genre`, `note`, `rating` e `author`, livros com e sem `pages`, e itens `want` sem nenhuma data.

Como os exemplos foram capturados

- `dart run tool/capture.dart`, dentro de `server/`. O script sobe o servidor (o mesmo código de `bin/server.dart`) numa porta livre, sem a latência padrão, e faz 44 requisições HTTP de verdade, gravando cada pedido e cada resposta em `tool/capture.json`.
- Os exemplos deste documento vêm da captura de 2026-11-06. Os tokens aparecem como `<token>`, e o `addedAt` dos itens criados é o horário daquela execução.
- Os números do resumo, os totais e a ordem da lista conferem com um cálculo feito à parte a partir da tabela do seed.